MITRE-CORE v3.0: Project Architecture & Dataset Guide

Date: 2026-03-11
Version: v3.0-TX (Transformer-Enhanced)

---

1. PROJECT OBJECTIVE

Primary Goal

Transform MITRE-CORE from an **O(n^2) Union-Find bottleneck** to an **O(n) transformer-candidate + deterministic Union-Find hybrid**, achieving:

- **3-4× speedup** over baseline at n=2,000 alerts
- **$0 cloud costs** (pure local training on RTX 5060 Ti 8GB)
- **Preserved determinism** (Union-Find backend unchanged)
- **85-90% recall** (vs 95% on A100, acceptable trade-off)

Core Problem

Current correlation engine uses O(n^2) pairwise scoring:

- n=1,000 → 500K comparisons (acceptable)
- n=2,000 → 2M comparisons (slow)
- n=10,000 → 50M comparisons (unusable)

Solution

Use transformer to pre-filter candidate edges from O(n^2) to O(n):

1. **Transformer** generates top-k candidate neighbors per alert (O(n))
2. **Union-Find** only scores candidate pairs, maintains exact transitive closure
3. **Result:** Near-linear time complexity with deterministic guarantees

---

2. ARCHITECTURE

2.1 High-Level Flow

```
Raw Alerts → Preprocessor → Transformer → Candidate Edges → Union-Find → Clusters
    ↓              ↓            ↓              ↓                ↓
  CSV/DF    AlertTokens   Affinity       Top-k Edges      Cluster IDs
            (512 max)     Scoring        (10 per alert)   + Metadata
```

2.2 Component Breakdown

A. Preprocessing Layer (`transformer/preprocessing/`)

AlertPreprocessor (`alert_preprocessor.py`)

- Converts pandas DataFrame → AlertToken objects
- Entity hashing (MD5-based, consistent across batches)
- Temporal bucketing (5-minute bins, 0-287 for 24h)
- Severity normalization (critical→1.0, info→0.1)

SlidingWindowBatcher (`sliding_window_batcher.py`)

- Creates overlapping windows (256 alerts max, 32-alert overlap)
- Time gap detection (>5min triggers hard break)
- Maintains continuity across batches

Output: GPU tensors ready for transformer

```
{
    'alert_ids': torch.Tensor [1, 256],        # Alert vocab indices
    'entity_ids': torch.Tensor [1, 256, 4],  # [src_ip, dst_ip, hostname,
username]
    'time_buckets': torch.Tensor [1, 256],     # 5-min bucket indices
    'attention_mask': torch.Tensor [1, 256],   # Valid positions
    'severity': torch.Tensor [1, 256],         # Normalized severity scores
}
```

B. Transformer Model (`transformer/models/`)

TransformerCandidateGenerator (`candidate_generator.py`)

```
Architecture (Optimized for 8GB GPU):
├── Embeddings
│   ├── alert_embedding (10000 × 128)
│   ├── entity_embedding (10000 × 128)
│   ├── time_embedding (288 × 128)
│   └── position_embedding (256 × 128)
├── 2× Transformer Layers (O(n) attention)
│   ├── Multi-head attention (4 heads, 32-dim each)
│   ├── Feed-forward (128 → 256 → 128)
│   └── LayerNorm + Residual connections
├── Biaffine Scorer (pairwise affinity)
│   └── W ∈ R^(128×128) learnable weights
└── Confidence Head
    └── Linear(128 → 64 → 1) + Sigmoid
```

Memory Budget (8GB):

Component	Size	Optimization

Component	Size	Optimization
Model weights	~500MB	FP16 precision
Activations	~2GB	Gradient checkpointing
Optimizer states	~1.5GB	CPU offload
Gradients	~500MB	FP16
Data batch	~1GB	Max 4 alerts
CUDA overhead	~2.5GB	System reserved
Total	**~8GB**	Fits exactly

C. Training Pipeline (`transformer/training/`)

GPUOptimizedTrainer (`gpu_trainer.py`)

```
Training Configuration:
- Mixed Precision: FP16 (cuts memory 50%)
- Gradient Accumulation: 16 steps (effective batch = 64)
- Gradient Checkpointing: Trade compute for memory
- CPU Offloading: Optimizer states in system RAM
- Learning Rate: 1e-4 with cosine annealing
- Loss: Contrastive (positive pairs same campaign)
```

Training Schedule (9-12 days on RTX 5060 Ti):

1. **Self-supervised pre-training (5-7 days):**
   - Dataset: All available alerts (UNSW + TON + CICAPT)
   - Task: Masked entity reconstruction + contrastive learning
   - Goal: Learn general alert representations
2. **Supervised fine-tuning (3-4 days):**
   - Dataset: Linux_APT (has campaign labels)
   - Task: Predict campaign pairs
   - Loss: Margin ranking + binary cross-entropy
   - Goal: Optimize for correlation accuracy

D. Union-Find Integration (`core/correlation_pipeline_v3.py`)

TransformerHybridPipeline

```
Flow:
1. Preprocess → AlertTokens
2. Transformer.generate_candidates(top_k=10)
3. Filter by threshold (score >= 0.5)
```

```
4. Pass candidates to Union-Find
5. Union-Find only unions candidate pairs
6. Return clusters with metadata
```

Key Property: Union-Find backend is unchanged. It still maintains exact transitive closure. The transformer only reduces the search space from $O(n^2)$ to $O(k)$ where $k = n \times top_k$.

---

3. DATASET ANALYSIS

3.1 Current Dataset Inventory

Dataset	Status	Size	Format	Labels	Utility
UNSW-NB15	☑ Active	0.07 GB	CSV	attack_cat	**Primary training**
NSL-KDD	☑ Active	0.05 GB	CSV	label	**Secondary training**
Linux_APT	⚠ Small	0.01 GB	Parquet	campaign	**Gold standard**
TON_IoT	☑ Active	0.03 GB	Parquet	MalwareIntelAttackType	**Temporal patterns**
CICIDS2017	✕ Empty	0 GB	N/A	N/A	**Delete**
CICAPT-IIoT	☑ Active	9.19 GB	CSV	Provenance logs	**Large-scale training**
Datasense_IIoT	☑ Active	1.01 GB	CSV	attack/benign	**Modern patterns**
Yokohama (YNU-IoTMal)	✕ Empty	0 GB	N/A	N/A	**Delete**

3.2 Dataset Requirements for Transformer Training

For effective transformer training, we need datasets with:

Required:

1. **Multi-stage campaigns** - Attack sequences that unfold over time
2. **Temporal sequencing** - Timestamps to learn temporal patterns
3. **Entity relationships** - Users, IPs, hostnames that link alerts

Preferred: 4. **MITRE ATT&CK tactic labels** - Ground truth for supervised learning 5. **Campaign/cluster labels** - For direct correlation training 6. **Sufficient volume** - >10K alerts for meaningful training

3.3 Dataset Recommendations

KEEP - Essential for Training:

1. UNSW-NB15 (0.07 GB)

- **Use:** Self-supervised pre-training
- **Strengths:** Large volume, diverse attack types, clean structure
- **Limitations:** No campaign labels (use attack_cat as proxy)
- **Preprocessing:** Filter to 10 attack categories, temporal sort

2. CICAPT-IIoT (9.19 GB)

- **Use:** Primary pre-training corpus
- **Strengths:** Massive scale (9GB), provenance tracking, modern attacks
- **Limitations:** Large file size requires chunking
- **Preprocessing:** Process in 256-alert windows, sliding overlap

3. TON_IoT (0.03 GB)

- **Use:** Temporal pattern learning
- **Strengths:** IoT-specific, temporal sequences, entity relationships
- **Limitations:** Smaller volume
- **Preprocessing:** Sort by EndDate, extract entity chains

4. Linux_APT (0.01 GB)

- **Use:** Supervised fine-tuning (gold standard)
- **Strengths:** Has campaign labels (ground truth), APT sequences
- **Limitations:** Very small (59 alerts)
- **Preprocessing:** Use for validation, data augmentation recommended

5. Datasense_IIoT_2025 (1.01 GB)

- **Use:** Modern attack pattern training
- **Strengths:** Recent (2025), temporal sequences, attack/benign labels
- **Preprocessing:** Extract attack sequences, 1-5 sec window samples

DELETE - Not Useful:

1. CICIDS2017 (0 GB)

- **Status:** Empty directory
- **Action:** Delete folder

2. Yokohama/YNU-IoTMal 2026 (0 GB)

- **Status:** Empty directory (arm/ folder empty)
- **Action:** Delete folder

3. NSL-KDD (0.05 GB)

- **Status:** Has data but outdated (1999 attacks)
- **Issue:** Attack patterns don't reflect modern threats

- **Action:** Archive or delete (optional)

3.4 Recommended Dataset Pipeline

```
Training Data Strategy:

Phase 1: Self-Supervised Pre-training (5-7 days)
├── 70% CICAPT-IIoT (chunked, 256-alert windows)
├── 15% UNSW-NB15 (shuffled, attack-type balanced)
└── 15% Datasense_IIoT (temporal sequences)

Phase 2: Supervised Fine-tuning (3-4 days)
├── 50% Linux_APT (campaign labels - gold standard)
├── 30% TON_IoT (entity chains)
└── 20% Synthetic campaigns (from UNSW patterns)

Phase 3: Validation
├── Linux_APT (held-out campaigns)
└── Synthetic test sets (controlled complexity)
```

---

4. ARCHITECTURE VALIDATION

4.1 Complexity Analysis

Original (v2.x):

- Time: $O(n^2)$ pairwise comparisons
- Space: O(n) for Union-Find structure
- n=2,000: 2M comparisons, ~2s latency

Transformer-Hybrid (v3.0):

- Time: $O(n \times d^2)$ where d=128 (fixed) for transformer + O(k) for UF
- Space: O(n) model weights + O(k) candidate edges
- n=2,000: ~100K candidates (top-10 per alert), ~150ms transformer + 100ms UF
- **Speedup:** 3-4× faster

4.2 Determinism Guarantee

Critical Property: Union-Find maintains exact transitive closure.

Proof:

- If A→B and B→C are in candidate list, Union-Find will union(A,B) and union(B,C)
- Transitive closure ensures A, B, C in same cluster
- Transformer only affects candidate selection, not clustering semantics
- If transformer misses an edge, worst case = fall back to $O(n^2)$ for that pair

---

5. EXECUTION CHECKLIST

Before Training:

- ☐ Delete empty dataset folders (CICIDS2017, Yokohama)
- ☐ Archive outdated datasets (NSL-KDD optional)
- ☐ Verify CICAPT-IIoT chunking works (9GB file)
- ☐ Create train/val split from Linux_APT

During Training:

- ☐ Monitor GPU memory (<7.5GB to avoid OOM)
- ☐ Checkpoint every 500 steps (~1 hour)
- ☐ Log training loss every 100 steps
- ☐ Validate on Linux_APT every epoch

After Training:

- ☐ Run v3_validation_suite.py
- ☐ Run v3_benchmarks.py
- ☐ Compare ARI vs v2.x baseline
- ☐ Test latency at n=2K

---

6. EXPECTED OUTCOMES

Performance Targets:

Metric	Target	v2.x Baseline
Latency (n=2K)	<150ms	~800ms
ARI (Linux_APT)	>0.35	0.4042 (HGNN-only)
Recall	>85%	90% (HGNN)
GPU Memory	<8GB	N/A
Training Time	9-12 days	N/A

Deliverables:

1. Trained model: `transformer_checkpoints/candidate_generator_v1.pt`
2. Validation report: `validation_results/v3_validation_report.json`
3. Benchmark results: `benchmarks/results/v3_benchmarks.json`
4. Integration: `core/correlation_pipeline_v3.py` (production-ready)

---

7. SUMMARY

Architecture: Transformer generates O(n) candidate edges, Union-Find maintains exact transitive closure on reduced search space.

Datasets: Use CICAPT-IIoT (9GB) + UNSW-NB15 + Datasense for pre-training. Use Linux_APT for supervised fine-tuning. Delete empty folders (CICIDS2017, Yokohama).

Training: 9-12 days on RTX 5060 Ti 8GB using FP16, gradient checkpointing, CPU offloading.

Outcome: 3-4× speedup, 85-90% recall, $0 cost, deterministic clustering preserved.